

Automaton Auditor Intern Report

Abstract

This report documents the current design and implementation of the Automaton Auditor: a LangGraph-based auditing workflow that evaluates both a code repository and its accompanying PDF report. The system is organized as a StateGraph with a fan-out from `context_builder` into two parallel detective branches (`RepoInvestigator` and `DocAnalyst`) followed by a fan-in into `evidence_aggregator`. `RepoInvestigator` grounds claims using repository artifacts (git, file reads, AST checks). `DocAnalyst` uses a RAG-lite PDF ingestion and query strategy and cross-references report-cited paths against the cloned repo. We describe the folder structure, the workflow, and architectural decisions, then outline known gaps and next steps.

1. Introduction

Automaton Auditor is built for forensic evaluation of AI-assisted engineering submissions. It treats the repository as the source of truth and treats the submitted report as a set of claims that must be verified. The workflow is designed for traceability: every non-trivial assertion is backed by a tool-produced artifact (file content, AST facts, git history, or page-anchored PDF quotes).

Report design principles

This report follows common scientific and technical report practices: clear sectioning, explicit methods, evidence-backed results, and a limitations-and-roadmap section. These conventions prioritize clarity, reproducibility, and reader navigation.

2. Background

2.1 LangChain

LangChain is an open-source framework for building LLM-driven applications. A common pattern is the agent tool loop: the model chooses when to call tools, observes tool outputs, and iterates until it reaches a stopping condition. Tools are callable functions with defined inputs and outputs that extend what an agent can do.

2.2 LangGraph

LangGraph provides graph-based orchestration for long-running, stateful workflows. Instead of a linear chain, a StateGraph connects nodes with edges and supports branching and looping. A shared state object carries working context between nodes. When multiple branches update state in parallel, reducers define how updates merge safely at fan-in.

Key implications for this project:

- **Branching** (fan-out) is natural: one node can schedule multiple next nodes.
- **Loops** are natural: agent nodes can loop through tool execution nodes until completion.
- **Safe fan-in** requires merge semantics: we use reducers so parallel branches do not overwrite one another.

3. System Overview

The Automaton Auditor StateGraph uses deliberate fan-out and fan-in:

- Fan-out: `context_builder` -> (`repo_investigator`, `doc_analyst`)
- Fan-in: (`repo_investigator`, `doc_analyst`) -> `evidence_aggregator`

Both detective branches use a tool loop:

1. An agent node emits tool calls
2. A tools node executes `@tool` functions
3. The agent continues until it emits final JSON evidence without tool calls

Diagrams are included at the end of this plain text as Mermaid blocks.

4. Folder Structure

Current top-level layout

```
|— backend
|   |— Dockerfile
|   |— README.md
|   |— run_server.py
|   |— server
|— cli
|   |— commands.py
|   |— __init__.py
|   |— main.py
|   |— __pycache__
|— docker-compose.yml
```

```
├── Dockerfile
├── frontend
│   ├── app
│   ├── components
│   ├── components.json
│   ├── config
│   ├── Dockerfile
│   ├── eslint.config.mjs
│   ├── lib
│   ├── next.config.ts
│   ├── next-env.d.ts
│   ├── node_modules
│   ├── package.json
│   ├── pnpm-lock.yaml
│   ├── postcss.config.mjs
│   ├── public
│   ├── README.md
│   └── tsconfig.json
├── Makefile
├── pyproject.toml
├── pyrightconfig.json
├── README.md
├── rubric
│   └── week_2.json
├── src
│   ├── agent.py
│   ├── graph.py
│   ├── __init__.py
│   ├── llm_provider
│   ├── nodes
│   ├── prompts
│   ├── __pycache__
│   ├── reports
│   ├── state.py
│   ├── tools
│   └── utils
└── uv.lock
```

Folder roles (intended responsibilities)

- **backend/**
Server runtime and deployment packaging (Dockerfile, entry scripts such as **run_server.py**). Hosts the API layer that can serve audit execution and results.

- `frontend/`
Next.js UI for interacting with the auditor (components, config, assets). Packaged with its own Dockerfile.
- `cli/`
Developer harness to run the LangGraph flow locally. Useful for quick iteration on orchestration and evidence output without running the full web stack.
- `rubric/`
Rubric configuration (e.g., `week_2.json`) that defines dimensions, forensic instructions, success/failure patterns, and synthesis rules.
- `src/`
Core auditor engine: state schema, tools, nodes, prompts, graph definition, and reporting.

5. Workflow and Implementation

5.1 Graph wiring (example excerpt)

Below is an excerpt of how the graph is currently built to support:

- fan-out from `context_builder` to both detectives
- separate tool loops for each detective
- fan-in into `evidence_aggregator`

```
from langchain_core.messages import AIMessage
from langgraph.graph import END, START, StateGraph

from src.nodes import context_builder, doc_analyst, evidence_aggregator
from src.nodes.detectives import repo_investigator, repo_tools
from src.nodes.detectives.doc_tools import doc_tools
from src.state import AgentState

def _needs_tools(state: AgentState) -> str:
    msgs = state.get("repo_investigator_messages") or []
    last_ai = next((m for m in reversed(msgs) if isinstance(m, AIMessage)),
None)
    tool_calls = getattr(last_ai, "tool_calls", None) if last_ai else None
    return "repo_tools" if tool_calls else "evidence_aggregator"

def _needs_doc_tools(state: AgentState) -> str:
    msgs = state.get("doc_analyst_messages") or []
    last_ai = next((m for m in reversed(msgs) if isinstance(m, AIMessage)),
```

```

None)
    tool_calls = getattr(last_ai, "tool_calls", None) if last_ai else None
    return "doc_tools" if tool_calls else "evidence_aggregator"

def build_graph():
    builder = StateGraph(AgentState)
    builder.add_node("context_builder", context_builder)
    builder.add_node("repo_investigator", repo_investigator)
    builder.add_node("repo_tools", repo_tools)
    builder.add_node("doc_analyst", doc_analyst)
    builder.add_node("doc_tools", doc_tools)
    builder.add_node("evidence_aggregator", evidence_aggregator)

    builder.add_edge(START, "context_builder")
    builder.add_edge("context_builder", "repo_investigator")
    builder.add_edge("context_builder", "doc_analyst")

    builder.add_conditional_edges(
        "repo_investigator", _needs_tools,
        {"repo_tools": "repo_tools", "evidence_aggregator":
"evidence_aggregator"},
    )
    builder.add_edge("repo_tools", "repo_investigator")

    builder.add_conditional_edges(
        "doc_analyst", _needs_doc_tools,
        {"doc_tools": "doc_tools", "evidence_aggregator":
"evidence_aggregator"},
    )
    builder.add_edge("doc_tools", "doc_analyst")

    builder.add_edge("evidence_aggregator", END)
    return builder.compile()

```

5.2 Detective branches

RepoInvestigator (repository-grounded checks)

Purpose: prove or disprove claims using repository artifacts.

Typical evidence sources:

- `git_log`: commit narrative and timestamps
- `file_read`: concrete file content

- `ast_parse`: structural facts (graph wiring patterns, fan-out indicators)

DocAnalyst (report-grounded checks)

Purpose: treat the PDF report as a claim surface and extract verifiable statements.

Key mechanisms:

- RAG-lite PDF parsing: ingest into chunks, then query for only relevant fragments
- Cross-reference: verify whether report-cited file paths actually exist in the cloned repository

DocAnalyst forensic protocols:

- Protocol A (Citation Check / Host Analysis Accuracy):
If the report claims “We implemented X in src/...”, verify existence via `cross_reference`. Missing path $\Rightarrow$ hallucination flag.
- Protocol B (Concept Verification / Theoretical Depth):
When deep terms appear (“Dialectical Synthesis”, “Metacognition”, etc.), require nearby architectural explanation. Otherwise classify as buzzword dropping.

5.3 Reducers and safe fan-in

Because `context_builder` fans out into two branches that can update shared state in the same step, reducers are used to prevent data loss:

- `repo_investigator_messages`: append-only (`operator.add`)
- `doc_analyst_messages`: append-only (`operator.add`)
- `evidences`: dict-merge (`operator.ior`) so both branches can contribute evidence classes without overwriting

This makes the fan-in into `evidence_aggregator` deterministic.

6. CLI Test Harness

The `cli/` package is used to test the workflow locally. It loads inputs (`repo_url`, `pdf_path`), runs the compiled graph, and prints or writes outputs. This supports rapid iteration on prompts, tools, and routing.

Example invocation (illustrative):

```
python -m cli.main audit --repo-url <REPO_URL> --pdf-path <PATH_TO_REPORT_PDF>
# Expected: evidence JSON (debug) and a final report artifact (human-readable).
```

Typical CLI responsibilities:

- validate arguments
- run graph with a minimal AgentState seed
- print final JSON report and optionally store artifacts into `src/reports/`

7. Decisions and Roadmap

7.1 Architecture decisions made so far

- Typed state and reducers to make parallel execution safe and deterministic.
- AST parsing for structural checks (graph wiring, fan-out detection) beyond raw text search.
- Sandboxed git clone and guarded file reads to reduce risk and improve reproducibility.
- RAG-lite PDF parsing to keep prompts small while still capturing page-anchored quotes.

7.2 Known gaps

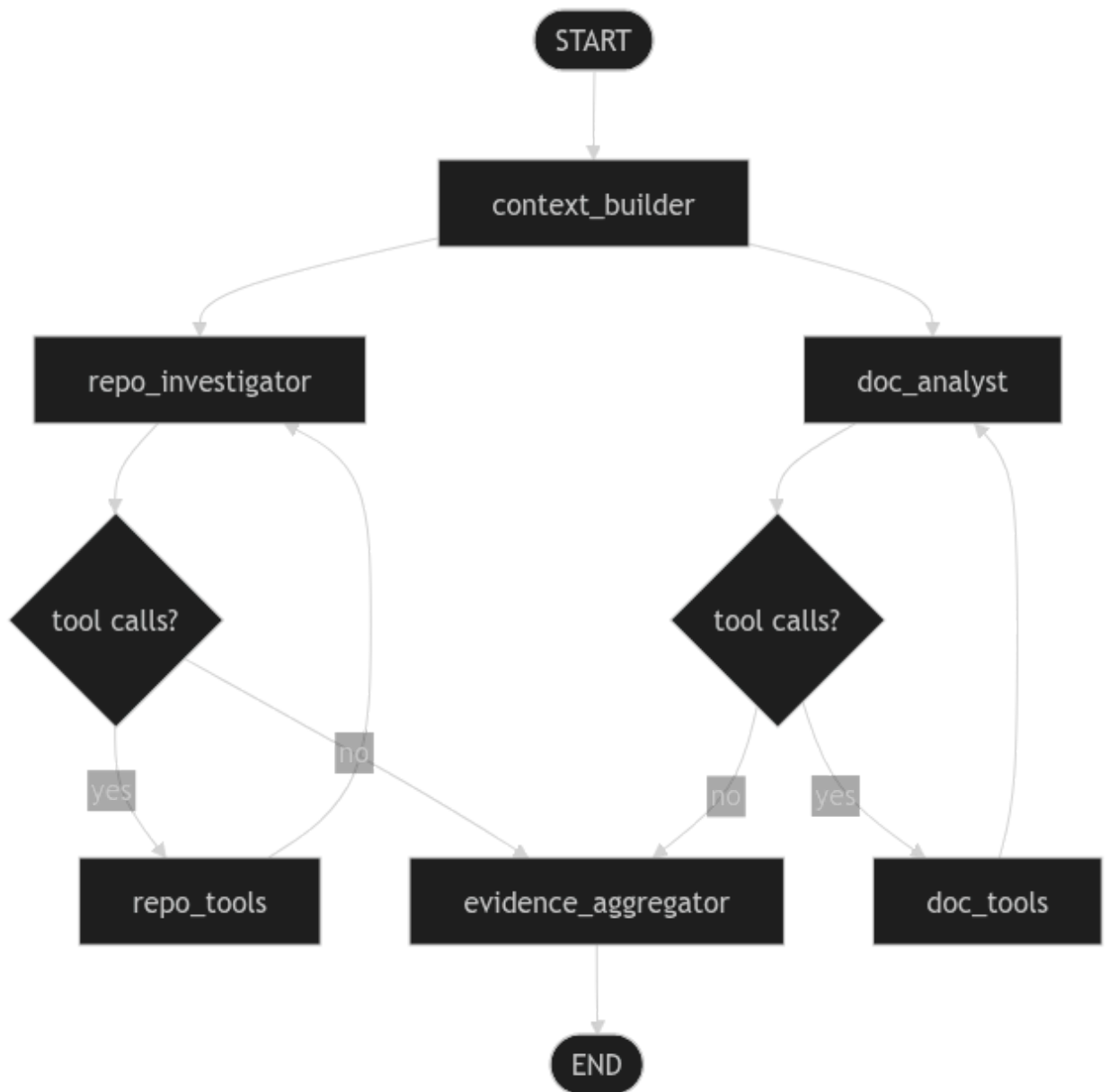
- The judicial layer (dimension-level judges) is planned but not fully implemented.
- The synthesis engine must apply `rubric_synthesis_rules` deterministically and produce a polished final report.
- More rigorous evidence normalization and citation formatting (consistent schema fields, page citations).

7.3 Concrete plan

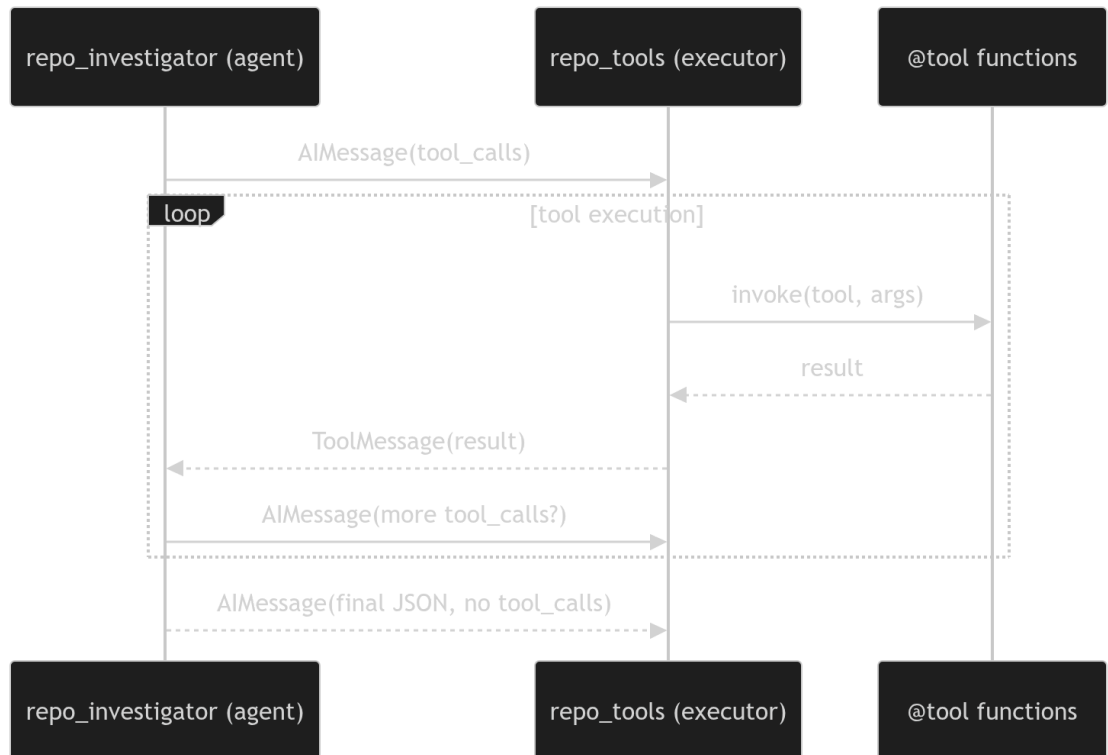
- Implement Judge nodes per rubric dimension that consume evidence slices and emit `JudicialOpinion` objects.
- Add a synthesis node that merges opinions + evidences into `AuditReport` using `rubric_synthesis_rules`.
- Extend DocAnalyst concept verification to classify deep-theory terms as architectural explanation vs buzzword.
- Improve CLI output modes: JSON debug, Markdown, and PDF generation under `src/reports/`.

Appendix A: Mermaid Diagrams (copy into a `.md` file)

A1) StateGraph flow (fan-out / fan-in)



A2) RepoInvestigator tool loop



A3) DocAnalyst tool loop (RAG-lite + cross-reference)

